

Guide to the Bash `trap` Command

1. Introduction to `trap`

The `trap` command in Bash allows you to specify commands or actions to be executed when the shell receives specific signals or encounters certain events. It is used for handling cleanups, managing interruptions, or debugging.

Syntax

```
trap 'commands' SIGNAL
```

- `commands`: The actions to perform when the signal is received.
- `SIGNAL`: The signal name (worded) or number (numeric).

Signals are used by the operating system to communicate with processes. For example, pressing `Ctrl+C` sends a `SIGINT` signal to terminate a script.

2. Common Usage Examples

1. Cleanup Temporary Files on Exit

Remove temporary files when the script exits:

```
trap 'rm -f /tmp/my_temp_file' EXIT
```

2. Handle Interruptions (`SIGINT`)

Handle `Ctrl+C` gracefully:

```
trap 'echo "Script interrupted!"; exit' SIGINT
```

3. Ignore Signals

Prevent interruption by ignoring `SIGINT`:

```
trap "" SIGINT
```

4. Debugging with `DEBUG`

Print each command before execution:

```
trap 'echo "Executing line: $LINENO"' DEBUG
```

3. Unsetting Traps

To reset a trap to its default behavior, use a hyphen (`-`) by one or more signal names.

Syntax

```
trap - SIGNAL [SIGNAL2 ...]
```

This command restores the default action for the specified signals.

Example

This example sets a custom handler for `SIGINT` and `SIGHUP`, and then later removes the handler for `SIGINT`, restoring its default behavior (terminating the script).

```
# Set a trap for SIGINT and SIGHUP
trap 'echo "Caught a signal, ignoring it."' \
    SIGINT SIGHUP

# ... some commands where Ctrl+C is ignored ...

# Unset the trap for SIGINT.
# SIGHUP is still trapped.
trap - SIGINT

# ... some other commands where Ctrl+C will
# now work as usual ...
```

4. Common Signals and Use Cases

Here is a detailed look at the most common signals used with `trap` in shell scripting.

EXIT (Pseudo-Signal)

- **Number:** N/A
- **Description:** This is a pseudo-signal that is triggered when the shell process itself exits.
- **Typical Use Case:** Performing cleanup tasks, such as deleting temporary files or logging the script's completion, regardless of how the script exits.

```
cleanup() {
    echo "Performing cleanup operations..."
    rm -f /tmp/temp_*.log
}
trap cleanup EXIT
```

SIGHUP (Hangup)

- **Number:** 1
- **Description:** Sent to a process when its controlling terminal is closed.
- **Typical Use Case:** Reloading configuration files without stopping a long-running service or daemon.

```
reload_config() {
    echo "SIGHUP received. Reloading configuration..."
    # ... code to reload config file ...
}
trap reload_config SIGHUP
```

SIGINT (Interrupt)

- **Number:** 2
- **Description:** Sent by the user pressing `Ctrl+C` in the terminal.
- **Typical Use Case:** Gracefully terminating a script, allowing it to perform cleanup or save state before exiting.

```
handle_interrupt() {  
    echo "Interrupted! Saving progress..."  
    # ... code to save progress ...  
    exit 1  
}  
trap handle_interrupt SIGINT
```

SIGQUIT (Quit)

- **Number:** 3
- **Description:** Sent by the user pressing `Ctrl+\` in the terminal.
- **Typical Use Case:** Similar to `SIGINT`, but often used to trigger a more forceful shutdown, including generating a core dump for debugging purposes.

```
handle_quit() {  
    echo "Quit signal received. Forcing exit..."  
    exit 1  
}  
trap handle_quit SIGQUIT
```

SIGTERM (Terminate)

- **Number:** 15
- **Description:** The default signal sent by the `kill` command. It requests a process to terminate.
- **Typical Use Case:** A standard way to ask a program to exit. A well-behaved script will trap `SIGTERM` to perform a clean shutdown.

```
handle_term() {  
    echo "Termination request received. Shutting down..."  
    # ... cleanup code ...  
    exit 0  
}  
trap handle_term SIGTERM
```

ERR (Pseudo-Signal)

- **Number:** N/A
- **Description:** A pseudo-signal that is triggered whenever a command fails (exits with a non-zero status).
- **Typical Use Case:** Error logging and script abortion on failure.

```
handle_error() {  
    local exit_code=$?  
    echo "Error on line $LINENO. Command exited with code $exit_code."  
    exit $exit_code  
}  
trap handle_error ERR
```

DEBUG (Pseudo-Signal)

- **Number:** N/A
- **Description:** A pseudo-signal that is triggered before every simple command is executed.
- **Typical Use Case:** Debugging scripts by printing each command before it runs, or inspecting variables.

```
trap 'echo "Executing: $BASH_COMMAND"' DEBUG
```

5. Complete Signal Reference Table

The following table provides a comprehensive list of signals.

Signal Name	Number	Description
EXIT	N/A	Executed when the script exits.
SIGHUP	1	Hangup detected on controlling terminal.
SIGINT	2	Interrupt from keyboard (Ctrl+C).
SIGQUIT	3	Quit from keyboard (Ctrl+).
SIGILL	4	Illegal instruction.
SIGTRAP	5	Trace/breakpoint trap.
SIGABRT	6	Abort signal from abort(3).
SIGBUS	7	Bus error (bad memory access).
SIGFPE	8	Floating-point exception.
SIGKILL	9	Kill signal (cannot be trapped).
SIGUSR1	10	User-defined signal 1.
SIGSEGV	11	Segmentation fault.
SIGUSR2	12	User-defined signal 2.
SIGPIPE	13	Broken pipe: write to pipe with no
		readers.
SIGALRM	14	Alarm clock.
SIGTERM	15	Termination signal.
SIGSTKFLT	16	Stack fault on coprocessor.
SIGCHLD	17	Child stopped or terminated.
SIGCONT	18	Continue if stopped.
SIGSTOP	19	Stop process (cannot be trapped).
SIGTSTP	20	Stop typed at terminal (Ctrl+Z).
SIGTTIN	21	Background process reads from TTY.
SIGTTOU	22	Background process writes to TTY.

SIGURG	23	Urgent condition on socket.
SIGXCPU	24	CPU time limit exceeded.
SIGXFSZ	25	File size limit exceeded.
SIGVTALRM	26	Virtual alarm clock.
SIGPROF	27	Profiling timer expired.
SIGWINCH	28	Window size change.
SIGPOLL	29	Pollable event.
SIGPWR	30	Power failure.
SIGSYS	31	Bad system call.
DEBUG	N/A	Triggered before every command execution.
ERR	N/A	Triggered when a command fails (non-zero
		exit).

6. Additional Notes

- `SIGKILL` and `SIGSTOP` cannot be trapped as they are handled directly by the kernel.
- Combining `trap` with functions provides better modularity and clarity.

```
cleanup() {
    echo "Performing cleanup..."
    rm -f /tmp/tempfile
}

trap cleanup EXIT
```